

Cards to rows: the Oscars, one step at a time

Chapter 6, Strategy 6: the loop from Wednesday's slide, built one piece at a time on the 98th Academy Awards page

The page for the 98th Academy Awards (<https://www.oscars.org/oscars/ceremonies/2026>) lists 24 award categories. It has no `<table>`. Each category is a *card*: a name, then one block for each honoree, first the winner and then the nominees. Each honoree block names a person or group and a film (Figure 1). This handout turns the cards into a pandas DataFrame with one row for each honoree:

category	honoree_type	entity	film
Actor in a Leading Role	Winner	Michael B. Jordan	Sinners
Actor in a Leading Role	Nominees	Timothée Chalamet	Marty Supreme
... 125 rows in all, from 24 cards			

Each step adds one thing to the step before it. Run each step in your notebook, and compare your output with the output here before you go to the next step.

- Code that you type.
- What Python shows back.
- 1 ... 7 The same seven parts of a card, in every figure and every step.

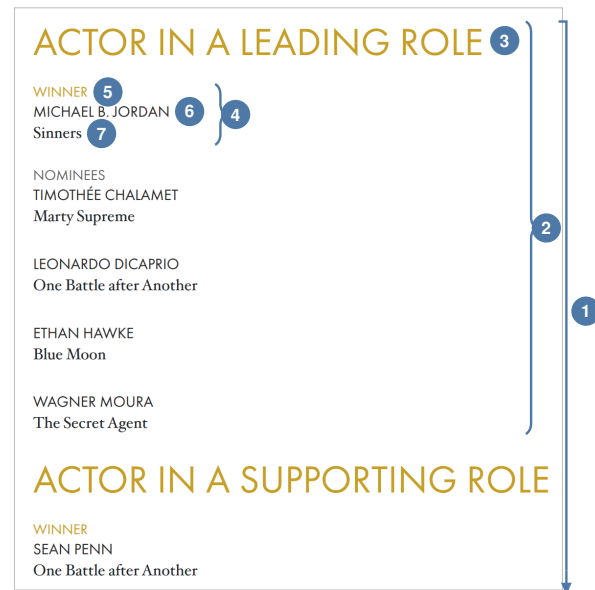


Figure 1. One card as the browser shows it, September 24, 2026. The numbers match Figure 2 in Step 3.

1 Ask for the page, and say who you are

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://www.oscars.org/oscars/ceremonies/2026"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
response = requests.get(url, headers=headers)
response.status_code
200
```

Put your own email address in place of `you@colorado.edu`. The `User-Agent` header tells the server who is asking; without it, `oscars.org` answers 403 (Forbidden). 200 means that the server sent the page.

2 Parse the HTML

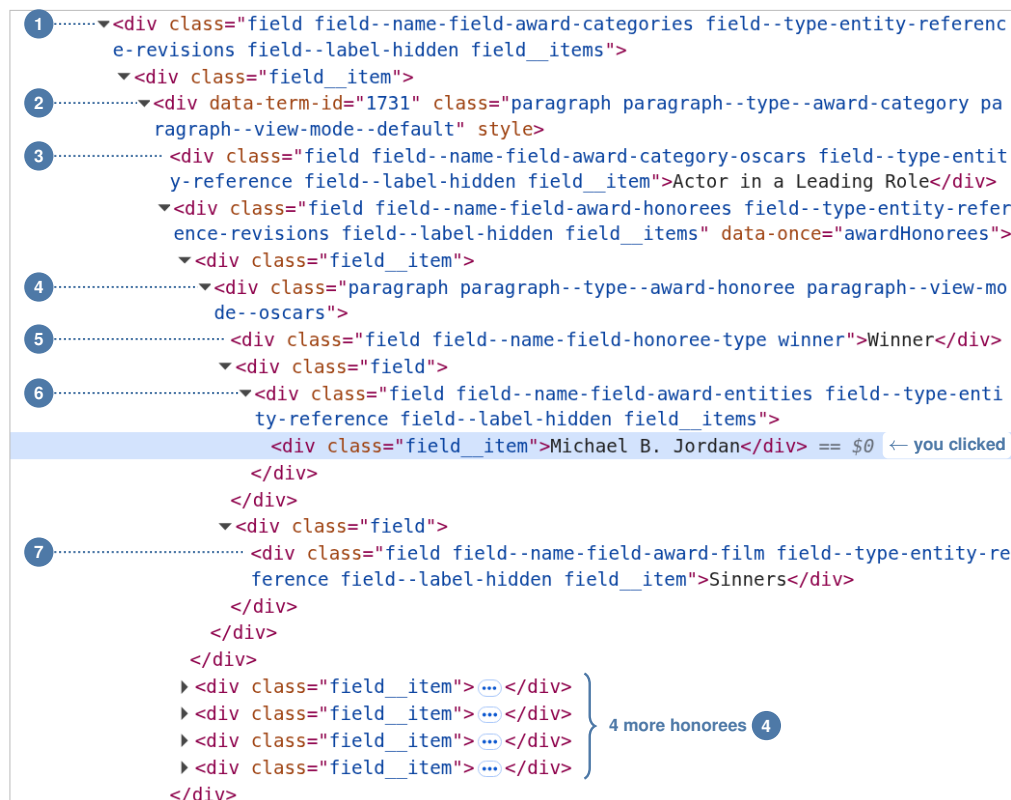
```
soup = BeautifulSoup(response.text, "html.parser")
soup.title.get_text()
'The 98th Academy Awards | 2026'
```

`soup` holds the page as a tree of tags that you can search. The title tells you that you have the ceremony page, not an error page.

3 Find one card in DevTools

Open the page in Chrome, right-click the name MICHAEL B. JORDAN, and select **Inspect**. The Elements panel opens with the `<div>` for that name selected (Figure 2). An indented line sits inside the nearest less-indented line above it. Read up from the selected line to find the containers around the name, and down to find the parts next to it. Click a triangle to open a collapsed line; Figure 2 has the film's `<div class="field">` open.

Look at the `class` attribute on each line. It holds several class names, separated by spaces. In Figure 2, the line at 5 has three names: `field`, `field--name-field-honoree-type`, and `winner`. `oscars.org` is built with Drupal, a content management system that makes most of these names from the names that the site gave its fields: `field--name-` plus a field name, and `paragraph--type--` plus a type of block. These names tell you what a `div` holds. Names such as `field` and `field__item` tell you only that the `div` is a box.



What each part holds

- 1 the area that holds all 24 cards
- 2 one card (a category)
- 3 the card's name
- 4 one honoree, a winner or a nominee
- 5 Winner or Nominees
- 6 the person or group (the *entity*)
- 7 the film

Figure 2. Chrome DevTools, Elements panel, after **Inspect** on MICHAEL B. JORDAN. September 24, 2026.

4 Match one class name, then count the matches

```
name_tag = soup.find("div", class_="field--name-field-award-category-oscars")
name_tag["class"]
['field',
 'field--name-field-award-category-oscars',
 'field--type-entity-reference',
 'field--label-hidden',
 'field__item']
```

BeautifulSoup keeps the `class` attribute as a list of names. `class_="..."` matches a tag when your name is any one item in that list, so you give only the one name that you need. (The underscore is necessary because `class` is a reserved word in Python.) Now count what each name matches on the whole page. The last item is the whole attribute from the line at 5, copied from DevTools:

```
for name in ["field", "field__item", "paragraph",
            "field--name-field-award-categories",
            "paragraph--type--award-category",
            "field--name-field-award-category-oscars",
            "paragraph--type--award-honoree",
            "field--name-field-honoree-type",
            "field--name-field-award-entities",
            "field--name-field-award-film",
            "field field--name-field-honoree-type winner"]:
    print(len(soup.find_all("div", class_=name)), name)
```

```
693 field
445 field__item
154 paragraph
1 field--name-field-award-categories
24 paragraph--type--award-category
24 field--name-field-award-category-oscars
125 paragraph--type--award-honoree
125 field--name-field-honoree-type
125 field--name-field-award-entities
125 field--name-field-award-film
25 field field--name-field-honoree-type winner
```

- 1
- 2
- 3
- 4
- 5
- 6
- 7

The general names match hundreds of divs: cards, honorees, and the rest of the page. The specific names match only what you want. Check each count against the page. The page shows 24 category headings, so 24 cards 2 is correct. There are 125 honorees 4 and 125 of each field 5 6 7, so each honoree has one of each. The whole attribute finds only the 25

winners: a string with spaces must match the attribute exactly, names and order. Select the one name that describes the content, then check its count.

5 Go to the area 1, then collect the cards 2

```
area = soup.find("div", class_="field--name-field-award-categories")
categories = area.find_all("div", class_="paragraph--type--award-category")
len(categories)
24
```

find returns the first match: one tag. find_all returns a list of all the matches. You call find_all on area, not on soup, so it searches only inside the awards area. categories is a list of the 24 cards, in the order of the page.

6 Take one card and read its name 3

```
cat = categories[0]
name_tag = cat.find(class_="field--name-field-award-category-oscars")
name_tag.get_text(strip=True)
'Actor in a Leading Role'
```

Build the steps for one card before you build them for 24. cat.find(...) searches only inside this card. This find has no "div", so it matches any tag with the class; every field on this page is a div, so both forms work. get_text(strip=True) gives the text inside the tag, without the spaces and line breaks around it.

7 Collect the honorees 4 in that one card

```
honorees = cat.find_all("div", class_="paragraph--type--award-honoree")
len(honorees)
5
```

In Step 4, this class name matched 125 divs on the whole page. Searched from cat, it matches the 5 in this card. Where you start a search decides what you get back. find_all also looks at every level below cat, so you do not step through the two wrapper divs between the card and its honorees (field--name-field-award-honorees and field__item in Figure 2).

8 Extract the three fields 5 6 7 from one honoree

```
h = honorees[0]
type_tag = h.find(class_="field--name-field-honoree-type")
entity_tag = h.find(class_="field--name-field-award-entities")
film_tag = h.find(class_="field--name-field-award-film")
print(type_tag.get_text(strip=True))
print(entity_tag.get_text(strip=True))
print(film_tag.get_text(strip=True))
Winner
Michael B. Jordan
Sinners
```

Each find starts from h, so it finds only the fields of this honoree. To see what strip=True removes, look at the raw text:

```
entity_tag.text
'\nMichael B. Jordan\n'
```

The line breaks come from the HTML around the inner field__item div. The type is also a class name (winner or nominee), but the text is what you want in the table.

9 Make one row for each honoree in the card

```
category_name = name_tag.get_text(strip=True)
rows = []
for h in honorees:
    rows.append({
        "category": category_name,
        "honoree_type": h.find(class_="field--name-field-honoree-type").get_text(strip=True),
        "entity": h.find(class_="field--name-field-award-entities").get_text(strip=True),
        "film": h.find(class_="field--name-field-award-film").get_text(strip=True),
    })
rows[1]
{'category': 'Actor in a Leading Role',
 'honoree_type': 'Nominees',
 'entity': 'Timothée Chalamet',
 'film': 'Marty Supreme'}
```

The three find calls from Step 8 go inside the loop, and their text goes into a dictionary: one dictionary per row, with the keys as column names. rows now holds 5 rows, and rows[1] is the second. category_name, from Step 6, puts the card's name

in each row. The page shows NOMINEES only once, but the HTML has the label in every nominee block; a script on the page hides the repeats (`style="display: none;"` in DevTools). requests does not run scripts, so each row gets its label.

10 Put the card loop inside a loop over all 24 cards

```
rows = []
for cat in categories:
    name_tag = cat.find(class_="field--name-field-award-category-oscars")
    if name_tag is None:
        continue
    category_name = name_tag.get_text(strip=True)
    for h in cat.find_all("div", class_="paragraph--type--award-honoree"):
        type_tag = h.find(class_="field--name-field-honoree-type")
        entity_tag = h.find(class_="field--name-field-award-entities")
        film_tag = h.find(class_="field--name-field-award-film")
        rows.append({
            "category": category_name,
            "honoree_type": type_tag.get_text(strip=True) if type_tag else None,
            "entity": entity_tag.get_text(strip=True) if entity_tag else None,
            "film": film_tag.get_text(strip=True) if film_tag else None,
        })
len(rows)
125
```

Two things are new. First, the outer loop `for cat in categories:` runs Steps 6 to 9 once for each card. `rows = []` moves above it, so all 24 cards add their rows to one list. Second, guards. `find` returns `None` when nothing matches, and calling `.get_text()` on `None` raises an `AttributeError` that stops the whole loop. `if name_tag is None: continue` skips a card that has no name, and ... `if type_tag else None` records a missing field as `None`. On the 2026 page no guard fires. The guards keep the loop running if one card is shaped differently.

11 Make the DataFrame, then check it against the page

```
df = pd.DataFrame(rows)
df.shape
(125, 4)
```

```
df.head()
```

	category	honoree_type	entity	film
0	Actor in a Leading Role	Winner	Michael B. Jordan	Sinners
1	Actor in a Leading Role	Nominees	Timothée Chalamet	Marty Supreme
2	Actor in a Leading Role	Nominees	Leonardo DiCaprio	One Battle after Another
3	Actor in a Leading Role	Nominees	Ethan Hawke	Blue Moon
4	Actor in a Leading Role	Nominees	Wagner Moura	The Secret Agent

```
df[["honoree_type"]].value_counts()
```

```
honoree_type
Nominees    100
Winner       25
Name: count, dtype: int64
```

25 winners for 24 categories looks like a bug in the loop. Look at the winners of one category:

```
df[(df["category"] == "Live Action Short Film") & (df["honoree_type"] == "Winner")]
```

	category	honoree_type	entity	film
70	Live Action Short Film	Winner	Sam A. Davis and Jack Piatt	The Singers
71	Live Action Short Film	Winner	Alexandre Singh and Natalie Musteata	Two People Exchanging Saliva

The category ended in a tie, the first Oscars tie since 2013, so the table is correct: check a surprising number against the page before you change your code. Also check what each column holds. You named the columns, but the site decides what goes in each field: for International Feature Film, `entity` is the film and `film` is the country (*Sentimental Value*, Norway); for Music (Original Song), `film` is the song title. Note these exceptions before you analyze the table.

The whole path. soup → area ① → each card ② → its name ③ and honorees ④ → each honoree's type ⑤, entity ⑥, and film ⑦ → one dictionary per row → `pd.DataFrame(rows)`. Checked against the live page on September 24, 2026. If your counts differ, the page has changed: inspect a card again and compare it with Figure 2.